

Physical Storage & Indexing Strategies.

Advanced Databases

M.Sc. Tran Quang Toan

Faculty of Data Science and Artificial Intelligence

Bases and coordinates

1. Physical Storage Fundamentals
2. Indexing - The Search Engine of DBMS
3. Advanced Optimization & Practice

Learning Objectives

By the end of this session, you will be able to:

- Explain how PostgreSQL organizes data on the physical disk (**Pages & Blocks**).
- Understand the internal mechanics of **B-Tree, Hash, and GIST** indexes.
- Differentiate between **Clustered** and **Non-Clustered** storage.
- Implement **Partial Indexes** to optimize storage and performance for AI datasets.

Physical Storage Fundamentals

Why Storage Matters for AI?

As AI engineers, why should we care about bytes on a disk?

- **I/O Bottleneck:** Disk access is $10^5\times$ slower than RAM. Training a model on data stored inefficiently leads to idle GPUs.
- **Data Locality:** Grouping related data together on disk minimizes "Disk Seek" time.
- **Vector/Large Data:** High-dimensional embeddings or JSON metadata require specialized storage (TOAST).

"Algorithms are fast, but data movement is expensive."

The Storage Hierarchy: How Postgres Sees the Disk



PostgreSQL does not interact with ”rows” directly on the disk; it uses a hierarchy:

1. **Database Cluster:** A collection of databases.
2. **Relation (Table):** Stored as a ”Heap File”.
3. **Page (Block):** The smallest unit of I/O.
   - Default size: **8 KB**.
   - Every time you read 1 row, Postgres loads the entire 8KB Page into the Buffer Pool.

Database Cluster: The Top Level

A **Database Cluster** is a single PostgreSQL instance running on a server.

- **Structure:** One cluster $\rightarrow$ Multiple Databases $\rightarrow$ Many Tables/Indexes.
- **Shared Resources:** All databases in a cluster share the same **RAM (Buffer Pool)**, CPU, and Disk I/O.

Practical Example

In production, you might have $\mathrm{app_db}$, $\mathrm{analytics_db}$, and $\mathrm{ai_db}$ on one cluster.

Discussion Question: *If one database in the cluster is under a heavy AI training workload, will the other databases be affected? Why?*

Relation (Table): The Heap File

In Postgres, a table is physically stored as a **Heap File**.

- **Unordered:** Rows are stored in no particular order.
- **Insertion:** Postgres inserts new data into any available free space.
- **MVCC Effect:** Updates do not overwrite; they create a new version of the row.

Consequence: Fragmentation

Over time, as rows are updated and deleted, the data becomes scattered ("fragmented") across the disk.

Discussion Question: *Why doesn't Postgres store rows sorted by Primary Key or Insertion time by default (like MySQL's InnoDB)?*

Page (Block): The Atomic Unit of I/O

The **Page** is the smallest unit of data that Postgres reads or writes.

- **The 8KB Rule:** Postgres never reads "half a page." To get 1 row, it must load the entire 8KB Page into RAM.
- **Performance Metric:** Query speed is determined by **how many pages** are read, not how many rows are returned.

```sql
SELECT name FROM users WHERE id = 1;
-- Result: Disk -> RAM: 1 full 8KB Page (containing multiple rows)
```

What happens during this query?

Analytical Challenge: *Which is faster: Scanning 1,000 rows stored in 10 pages, or scanning 100 rows scattered across 100 pages? Why?*

Anatomy of a Postgres Page

Each 8KB Page contains:

- **Page Header:** Metadata about the page (checksum, free space location).
- **Item Pointers:** Short pointers (4 bytes) to the start of each row.
- **Free Space (The Hole):** New rows are inserted here.
- **Tuples (Rows):** The actual data stored from the bottom up.

Optimization Tip: Keeping rows small allows more rows per page, reducing I/O operations.

From Table to Disk: The TID

Every row in Postgres has a hidden physical address called a **TID (Tuple ID)**.

- Format: $(\text{BlockNumber}, \text{Offset})$
- Example: $(42, 5)$ means the 5th row on the 42nd page.

```sql
-- Select the hidden ctid column
SELECT ctid, title FROM film LIMIT 5;
```

Querying the physical location of rows

Insight: Indexes do not store data; they store the **TID** that points to the Heap File.

Handling Large Data: TOAST

Since a Page is only 8KB, what happens if you store a 1MB PDF or a large AI Embedding in a column?

- **TOAST** kicks in automatically when a row exceeds 2KB.
- **Compression:** Postgres tries to compress the data first.
- **Out-of-line storage:** If still too big, Postgres moves the data to a separate "TOAST table" and leaves a pointer in the original row.

Result: Main table scans remain fast because "heavy" data is tucked away.

Indexing - The Search Engine of DBMS

Indexing: The ”Table of Contents” Strategy

An index is a separate data structure that allows the DBMS to find rows without scanning the entire table (**Sequential Scan**).

- **Pros:** Dramatic speedup for SELECT queries.
- **Cons:** Slows down INSERT/UPDATE/DELETE (Index must be updated).
- **Storage:** Indexes consume extra disk space.

Rule of Thumb

Index columns used in WHERE, JOIN, and ORDER BY.

B-Tree: The Gold Standard

- **Structure:** A self-balancing tree. All "Leaf Nodes" are at the same depth.
- **Performance:** $O(\log N)$ search time.
- **Usage:** Handles $<$, $<=$, $=$, $>=$, $>$ and $\mathrm{BETWEEN}$ operators.
- **Sorting:** Data in B-Tree leaves is sorted, making $\mathrm{ORDER\ BY}$ near-instant.

Deep Dive: B-Tree Traversal

How does Postgres find a value (e.g., $\mathrm{ID} = 45$)?

1. Start at the **Root Node**.
2. Compare 45 against the keys in the node to decide which **Internal Node** to follow.
3. Repeat until reaching the **Leaf Node**.
4. The Leaf Node contains the **TID** (Physical address).
5. Access the **Heap File** (Table) to fetch the actual row.

For a table with 1 million rows, a B-Tree usually only needs 3-4 levels.

Hash Index: Fast but Limited

- **Mechanism:** Uses a Hash Function to map values to "Buckets".
- **Pros:** Can be slightly faster than B-Tree for exact matches ($=$).
- **Cons:**
  - Does NOT support range queries ($\text{price} > 100$).
  - Does NOT support sorting.
  - Historically less reliable than B-Tree (fixed in Postgres 10+).

```sql
CREATE INDEX idx_customer_email_hash
ON customer USING HASH (email);
```

Creating a Hash Index

GIST: For ”Messy” Data

B-Tree is for linear data. **GIST** is a framework for complex objects:

- **Spatial Data:** Find all points within a polygon (PostGIS).
- **Full-Text Search:** Indexing documents.
- **AI Context:** ”Nearest Neighbor” (KNN) searches for feature vectors.

GIST allows ”lossy” indexing, which is highly efficient for high-dimensional data.

GIN: The Power of Multi-values

While B-Tree maps *1 Key* $\rightarrow$ *1 Row*, GIN maps *1 Value* $\rightarrow$ *Many Rows*.

- **Best for:** Arrays, JSONB, and Full-text search.
- **Example:** Finding all users who have the tag 'Python' in their skills array.

```sql
-- Indexing the entire JSONB structure
CREATE INDEX idx_metadata_gin ON raw_data USING GIN (metadata_jsonb);

-- Querying using the "contains" operator
SELECT * FROM raw_data WHERE metadata_jsonb @> '{"status": "active"}'
    ;
```

Indexing JSONB with GIN

Clustered vs. Non-Clustered Index

Feature	Clustered Index (MySQL)	Non-Clustered (Postgres)
Physical Order	Table rows follow Index order.	Table rows are in a Heap (unordered).
Data Location	Stored directly in Index leaves.	Index stores TIDs (pointers) to Heap.
Quantity	Only 1 per table.	Multiple indexes per table.
Update Cost	High (may require moving rows).	Lower (only update the index).

The CLUSTER Command

Postgres does not have "Automatic Clustering." However, you can manually re-order a table to speed up range scans.

```
-- 1. Create a normal B-Tree index
CREATE INDEX idx_film_title ON film(title);

-- 2. Physically re-order the table based on that index
CLUSTER film USING idx_film_title;
```

Clustering the film table

Warning

CLUSTER is a one-time operation. New INSERTs will be added to the end of the heap, and the table will become "un-clustered" over time.

The Dark Side: Maintenance & Bloat

Indexes are not free!

- **Write Overhead:** Every INSERT must update every index on the table.
- **Index Bloat:** When rows are deleted, Postgres leaves "dead tuples". If the index doesn't clean up properly, it grows indefinitely.
- **The Solution: VACUUM**. It reclaims space from dead tuples in both the Heap and Indexes.

Advice for AI Engineers: When bulk-loading millions of rows for training data, **DROP** the indexes first, load the data, then **RE-CREATE** them.

Advanced Optimization & Practice

Partial Indexes: Efficiency Redefined

Why index a million rows if you only ever query 5% of them? **Partial Index** uses a WHERE clause to filter which rows get indexed.

```sql
-- Only index rows where return_date is NULL (active rentals)
CREATE INDEX idx_unreturned_rentals
ON rental (inventory_id)
WHERE return_date IS NULL;
```

Scenario: Fast search for unreturned rentals

Benefits for AI Workloads:

- **Storage:** Huge savings when indexing "anomalies" or "active" subsets.
- **Performance:** Smaller tree depth = faster traversal.
- **Maintenance:** No overhead when updating rows that don't meet the criteria.

Comparison: Full Index vs. Partial Index

Consider a table with 10M rows, but only 10,000 are "Pending":

Metric	Full B-Tree Index	Partial Index (WHERE status='pending')
Index Entries	10,000,000	10,000
Disk Space	220 MB	0.5 MB
Insert Overhead	Hits every insert	Only hits if status is 'pending'
Query Speed	Fast	Ultra-fast (for specific queries)

Important

The query's `WHERE` clause must match the index's `WHERE` clause for the Optimizer to use it.

Multicolumn Indexes: The ”Leading Column” Rule



Postgres can index up to 32 columns in a single index.

```
CREATE INDEX idx_staff_name ON staff (last_name, first_name);
```

How it works:

- Data is sorted by $\mathrm{last_name}$, then by $\mathrm{first_name}$.
- **The Rule:** This index helps queries on $(\mathrm{last_name})$ or $(\mathrm{last_name}, \mathrm{first_name})$.
- **The Trap:** It is **useless** for a query filtering only by $(\mathrm{first_name})$.

Tip: Always put the column with the highest *cardinality* (most unique values) first.

Lab 1: Inspecting Table Size

Connect to the dvdrental database and run these diagnostics:

```sql
-- 1. Check table vs index size
SELECT
    relname AS object_name,
    pg_size_pretty(pg_table_size(fed.oid)) AS data_size,
    pg_size_pretty(pg_indexes_size(fed.oid)) AS index_size,
    pg_size_pretty(pg_total_relation_size(fed.oid)) AS total_size
FROM pg_class fed
JOIN pg_namespace n ON n.oid = fed.relnamespace
WHERE fed.relkind = 'r' AND n.nspname = 'public'
ORDER BY pg_total_relation_size(fed.oid) DESC;
```

Diagnostic Queries

Goal: Identify which tables in dvdrental are "Index Heavy".

Lab 2: The Impact of a New Index

Let's measure the physical growth of the database.

```
-- Step 1: Check current size of 'payment' indexes
SELECT pg_size_pretty(pg_indexes_size('payment'));

-- Step 2: Create a heavy index on payment_date
CREATE INDEX idx_payment_date_long ON payment (payment_date, amount,
    customer_id);

-- Step 3: Check size again
SELECT pg_size_pretty(pg_indexes_size('payment'));
```

Discussion: How many Pages (8KB) did this new index consume?

Lab 3: Partial Index Practice

Scenario: Our marketing team only queries "Expensive" films (Replacement cost > 25).

```sql
-- Create Partial Index
CREATE INDEX idx_expensive_films
ON film (replacement_cost)
WHERE replacement_cost > 25.00;

-- Find the size of this specific index
SELECT relname, pg_size_pretty(pg_relation_size(relid))
FROM pg_stat_user_indexes
WHERE relname = 'idx_expensive_films';
```

Task: Compare this size with a hypothetical full index on $\mathrm{replacement_cost}$.

Indexing Best Practices

- **Don’t Over-Index:** Each index adds latency to data ingestion.
- **Index Selectivity:** Only index columns where a value matches < 10
- **Monitor Bloat:** Use REINDEX or VACUUM FULL if an index grows too large after many deletes.
- **Covering Indexes:** Use INCLUDE to add extra data to the index leaves to avoid hitting the Heap File (**Index Only Scan**).

Index Selection Cheat Sheet

Scenario	Recommended Index	Why?
Standard ID Search	**B-Tree**	Default, balanced, fast.
Coordinates / Maps	**GIST**	Handles 2D/3D shapes.
JSONB Metadata	**GIN**	Indexes keys/values inside JSON.
Status = 'Active'	**Partial B-Tree**	Saves space, targets relevant data.
Unique Email Login	**B-Tree (Unique)**	Ensures no duplicates + fast lookups.

Conclusion

Q&A

Summary: We've moved from "Logical SQL" to "Physical Storage". Understanding Pages and Indexes is the foundation of Query Optimization.
Next Session: Query Optimization. We will learn how the "Brain" of Postgres (The Optimizer) chooses between a Seq Scan and an Index Scan using `EXPLAIN ANALYZE`.